

Delegation vs Inheritance in Java

[Inheritance in Java](#) programming is the process by which one class takes the property of another other class. i.e. the new classes, known as derived or child class, take over the attributes and behavior of the pre-existing classes, which are referred to as base classes or super or parent class.

Delegation is simply passing a duty off to someone/something else.

- Delegation can be an alternative to inheritance.
- Delegation means that you use an object of another class as an instance variable, and forward messages to the instance.
- It is better than inheritance for many cases because it makes you to think about each message you forward, because the instance is of a known class, rather than a new class, and because it doesn't force you to accept all the methods of the super class: you can provide only the methods that really make sense.
- Delegation can be viewed as a relationship between objects where one object forwards certain method calls to another object, called its delegate.
- The primary advantage of delegation is run-time flexibility – the delegate can easily be changed at run-time. But unlike inheritance, delegation is not directly supported by most popular object-oriented languages, and it doesn't facilitate [dynamic polymorphism](#).

When to use what?

Here are some examples when inheritance or delegation are being used:

Assume your class is called B and the derived/delegated to class is called A then **If**

- You want to express relationship (is-a) then you want to use Inheritance.
- You want to be able to pass your class to an existing API expecting A's then you need to use inheritance.
- You want to enhance A, but A is final and can no further be sub-classed then you need to use [composition](#) and delegation.
- You want functionality of a method and do not want to override the method then you should go for delegation.
- Inheritance ties your class into the hierarchy of classes you are inheriting from, whereas with delegation you may keep your place in your own hierarchy and delegate calls to another.